Project Report

Project Name: A Domain Specific Language (DSL) for Convolutional Neural Network (CNN)

Name: Abhijeet Deb Nath

Roll: 2107083

Course Name: Compiler Design Laboratory

Course Code: CSE 3212

Date: 30/03/2026

Abstract

NetLang is a domain-specific ahead-of-time compiler for fixed-shape CNN inference. It parses a compact neural-network DSL, performs scope-aware semantic analysis and tensor-shape inference, lowers source-level assignments into a dependency graph, applies a stack of tensor-oriented planner passes, and emits specialized C linked against a runtime and AVX2 kernel library.

The most important architectural distinction from a traditional compiler is that NetLang is not primarily concerned with control-flow normalization, scalar SSA, or register allocation. Its main algorithmic work is graph recovery, shape propagation, flatten aliasing, packed-weight preparation, activation-slot reuse, execution micro-tiling, and backend kernel selection.

1. Compiler and End-to-End Story

The strongest report narrative is to present NetLang as a planner-driven tensor compiler rather than as a small parser that happens to emit C. The pipeline in the repository is already layered in a way that supports that story: frontend parsing, semantic validation, graph lowering, planner construction, backend specialization, and native-oriented code emission.

The key claim this compiler can support is the following: under a restricted fixed-shape CNN language model, a compiler can perform stronger ahead-of-time specialization than a traditional general-purpose frontend because tensor shapes, graph structure, and many backend decisions are statically available before code emission.

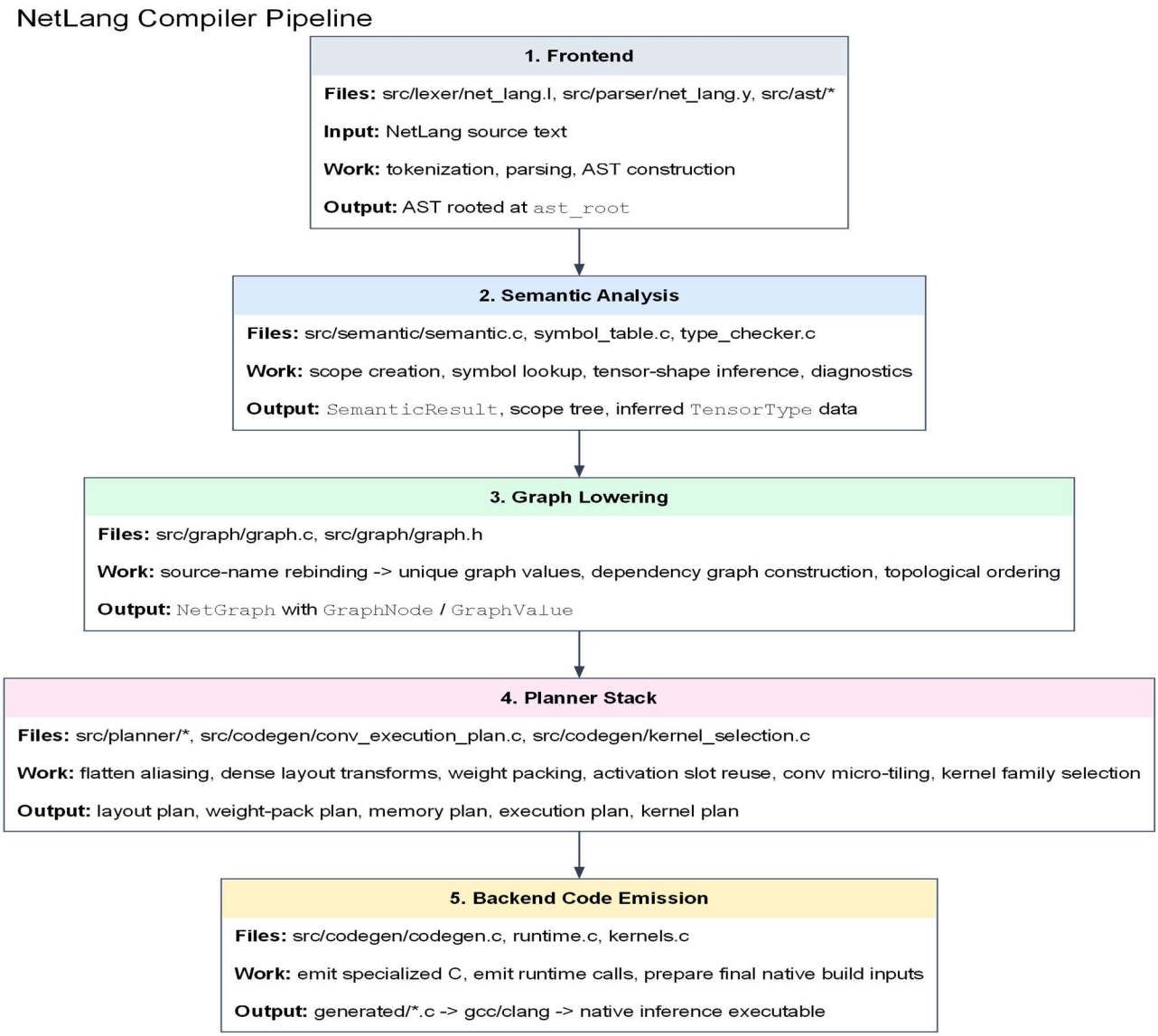

Figure 1. Implemented NetLang pipeline from source text to emitted specialized C and native build inputs.

2. Exact NetLang Surface Language and Formal Grammar

This section lists the exact surface strings, token classes, non-terminals, and production families implemented by the current lexer and parser. The goal is to keep the report technically precise and grounded in src/lexer/net_lang.l and src/parser/net_lang.y.

2.1 Terminal inventory

The lexer has two kinds of terminals: literal strings such as network, Conv2D, or relu; and regex-defined token classes such as IDENTIFIER and NUMBER. The following table enumerates all fixed surface strings returned as parser terminals.

Category	Surface string	Token
Keyword	network	NETWORK
Keyword	module	MODULE
Keyword	return	RETURN
Keyword	from	FROM
Keyword	input	INPUT
Keyword	shape	SHAPE
Keyword	weights	WEIGHTS
Layer constructor	Conv2D	CONV2D
Layer constructor	Dense	DENSE
Layer constructor	MaxPool	MAXPOOL
Layer constructor	AvgPool	AVGPOOL
Layer constructor	Flatten	FLATTEN
Layer constructor	Add	ADD
Layer constructor	Concat	CONCAT
Layer constructor	BatchNorm	BATCHNORM
Layer constructor	LayerNorm	LAYERNORM
Parameter keyword	filters	FILTERS
Parameter keyword	kernel	KERNEL
Parameter keyword	activation	ACTIVATION
Parameter keyword	stride	STRIDE
Parameter keyword	padding	PADDING
Parameter keyword	pool	POOL
Parameter keyword	units	UNITS
Activation literal	relu	RELU
Activation literal	sigmoid	SIGMOID
Activation literal	tanh	TANH
Activation literal	softmax	SOFTMAX

Activation literal	linear	LINEAR
Punctuation	{	LBRACE
Punctuation	}	RBRACE
Punctuation	[	LBRACKET
Punctuation	]	RBRACKET
Punctuation	(	LPAREN
Punctuation	)	RPAREN
Punctuation	:	COLON
Punctuation	,	COMMA
Punctuation	=	ASSIGN

2.2 Regex-defined token classes and ignored lexemes

Lexer name	Pattern	Behavior
INTEGER	[0-9]+	NUMBER
FLOAT	[0-9]+\.[0-9]*	FLOAT_NUM
ID	[a-zA-Z_][a-zA-Z0-9_]*	IDENTIFIER
STRING	"[^"]*"	STRING_LIT
WS	[\t\r]+	ignored
NEWLINE	\n	line/column bookkeeping
SLCOMMENT	//[^\n]*	ignored
/* ... */	multi-line comment state	ignored

2.3 Non-terminal inventory

The parser declares the following non-terminals. They correspond directly to the grammar's structural categories and are the grammar symbols from which the AST is built.

Non-terminal	Role
program	root non-terminal
network_def	top-level network definition
module_def	top-level module definition

network_body	input, optional weights, and statement body
module_list	helper modules before the network
module_params	module formal parameter list
param_list	comma-separated identifiers
input_decl	input(shape: [...])
weights_decl	weights("...")
optional_weights_decl	empty or explicit weights declaration
statement_list	possibly empty statement sequence
statement_list_nonempty	non-empty statement sequence
statement	currently assignment only
assignment	IDENTIFIER = layer_expr [from expr]
optional_from_clause	empty or FROM expr
return_stmt	RETURN expr
layer_expr	union of all layer-producing expressions
conv2d_layer	Conv2D(...)
dense_layer	Dense(...)
pool_layer	MaxPool(...) or AvgPool(...)
flatten_layer	Flatten()
add_layer	Add(x, y, ...)
concat_layer	Concat(x, y, ...)
norm_layer	BatchNorm() or LayerNorm()
module_call	Identifier(parameter map)
layer_params	optional named argument list
layer_param_list	comma-separated named arguments
add_args	comma-separated Add operands
concat_args	comma-separated Concat operands
expr	number, array, identifier, or activation literal
identifier_expr	IDENTIFIER or INPUT

number	NUMBER or FLOAT_NUM
array	[array_elements]
array_elements	comma-separated numeric elements
activation_value	relu | sigmoid | tanh | softmax | linear

2.4 Production rules

The following listings present the production structure implemented by the parser. The listings are normalized for report readability, but they preserve the same rule coverage as the Bison grammar.

```
program
    : network_def
    | module_list network_def
    ;

module_list
    : module_def
    | module_list module_def
    ;

network_def
    : NETWORK IDENTIFIER LBRACE network_body RBRACE
    ;

network_body
    : input_decl optional_weights_decl statement_list_nonempty
    ;

module_def
    : MODULE IDENTIFIER LPAREN module_params RPAREN
      LBRACE statement_list return_stmt RBRACE
    ;

module_params
    : /* empty */
    | param_list
    ;

param_list
    : IDENTIFIER
    | param_list COMMA IDENTIFIER
    ;

input_decl
```

```
    : INPUT LPAREN SHAPE COLON array RPAREN
    ;

weights_decl
    : WEIGHTS LPAREN STRING_LIT RPAREN
    ;

optional_weights_decl
    : /* empty */
    | weights_decl
    ;

statement_list
    : /* empty */
    | statement_list statement
    ;

statement_list_nonempty
    : statement
    | statement_list_nonempty statement
    ;

statement
    : assignment
    ;

assignment
    : IDENTIFIER ASSIGN layer_expr optional_from_clause
    ;

optional_from_clause
    : /* empty */
    | FROM expr
    ;

return_stmt
    : RETURN expr
    ;

layer_expr
    : conv2d_layer
    | dense_layer
    | pool_layer
    | flatten_layer
    | add_layer
    | concat_layer
```

```
    | norm_layer
    | module_call
    ;

conv2d_layer
    : CONV2D LPAREN FILTERS COLON expr COMMA
      KERNEL COLON array COMMA
      STRIDE COLON expr COMMA
      PADDING COLON expr COMMA
      ACTIVATION COLON activation_value RPAREN
    ;

dense_layer
    : DENSE LPAREN UNITS COLON expr COMMA
      ACTIVATION COLON activation_value RPAREN
    ;

pool_layer
    : MAXPOOL LPAREN POOL COLON array COMMA
      STRIDE COLON expr COMMA PADDING COLON expr RPAREN
    | AVGPOOL LPAREN POOL COLON array COMMA
      STRIDE COLON expr COMMA PADDING COLON expr RPAREN
    ;

flatten_layer
    : FLATTEN LPAREN RPAREN
    ;

add_layer
    : ADD LPAREN add_args RPAREN
    ;

add_args
    : expr
    | add_args COMMA expr
    ;

concat_layer
    : CONCAT LPAREN concat_args RPAREN
    ;

concat_args
    : expr
    | concat_args COMMA expr
    ;
```

```
norm_layer
    : BATCHNORM LPAREN RPAREN
    | LAYERNORM LPAREN RPAREN
    ;

module_call
    : IDENTIFIER LPAREN layer_params RPAREN
    ;

layer_params
    : /* empty */
    | layer_param_list
    ;

layer_param_list
    : IDENTIFIER COLON expr
    | FILTERS COLON expr
    | KERNEL COLON expr
    | ACTIVATION COLON expr
    | STRIDE COLON expr
    | PADDING COLON expr
    | POOL COLON expr
    | UNITS COLON expr
    | layer_param_list COMMA IDENTIFIER COLON expr
    | layer_param_list COMMA FILTERS COLON expr
    | layer_param_list COMMA KERNEL COLON expr
    | layer_param_list COMMA ACTIVATION COLON expr
    | layer_param_list COMMA STRIDE COLON expr
    | layer_param_list COMMA PADDING COLON expr
    | layer_param_list COMMA POOL COLON expr
    | layer_param_list COMMA UNITS COLON expr
    ;

expr
    : number
    | array
    | identifier_expr
    | activation_value
    ;

identifier_expr
    : IDENTIFIER
    | INPUT
    ;

number
```

```
    : NUMBER
    | FLOAT_NUM
    ;

array
    : LBRACKET array_elements RBRACKET
    ;

array_elements
    : number
    | array_elements COMMA number
    ;

activation_value
    : RELU
    | SIGMOID
    | TANH
    | SOFTMAX
    | LINEAR
    ;
```

2.5 Parse-time structural constraints

- filters and units must be positive integer literals
- kernel and pool must be exactly two-element integer arrays
- Conv2D stride must be a positive integer literal
- pooling stride must be a non-negative integer literal
- padding must be a non-negative integer literal
- input shape must later validate as a 3-element [H, W, C] array in semantic analysis

3. Frontend AST Design

The parser lowers syntax directly into a hand-written AST declared in src/ast/ast.h. The AST is deliberately compact. It has node families for top-level structure, statements, layer operations, module calls, and primitive expressions such as identifiers, numbers, strings, and arrays.

A particularly important design choice is that activation is represented as a field inside NODE_CONV2D and NODE_DENSE rather than as a separate AST node. This matters later because backend fusion is currently driven mostly by inline activation metadata instead of explicit Conv -> ReLU graph rewrites.

For the AST figure, a smaller network is more informative than LeNet-5 because the full tree can be shown completely rather than as a shallow excerpt. The AddBranch program is therefore used below as a complete parser output example.

```
network AddBranch {
    input(shape: [28, 28, 4])
    left = MaxPool(pool: [2, 2], stride: 2, padding: 0) from input
    right = AvgPool(pool: [2, 2], stride: 2, padding: 0) from input
    merged = Add(left, right)
    output = Flatten() from merged
}
```

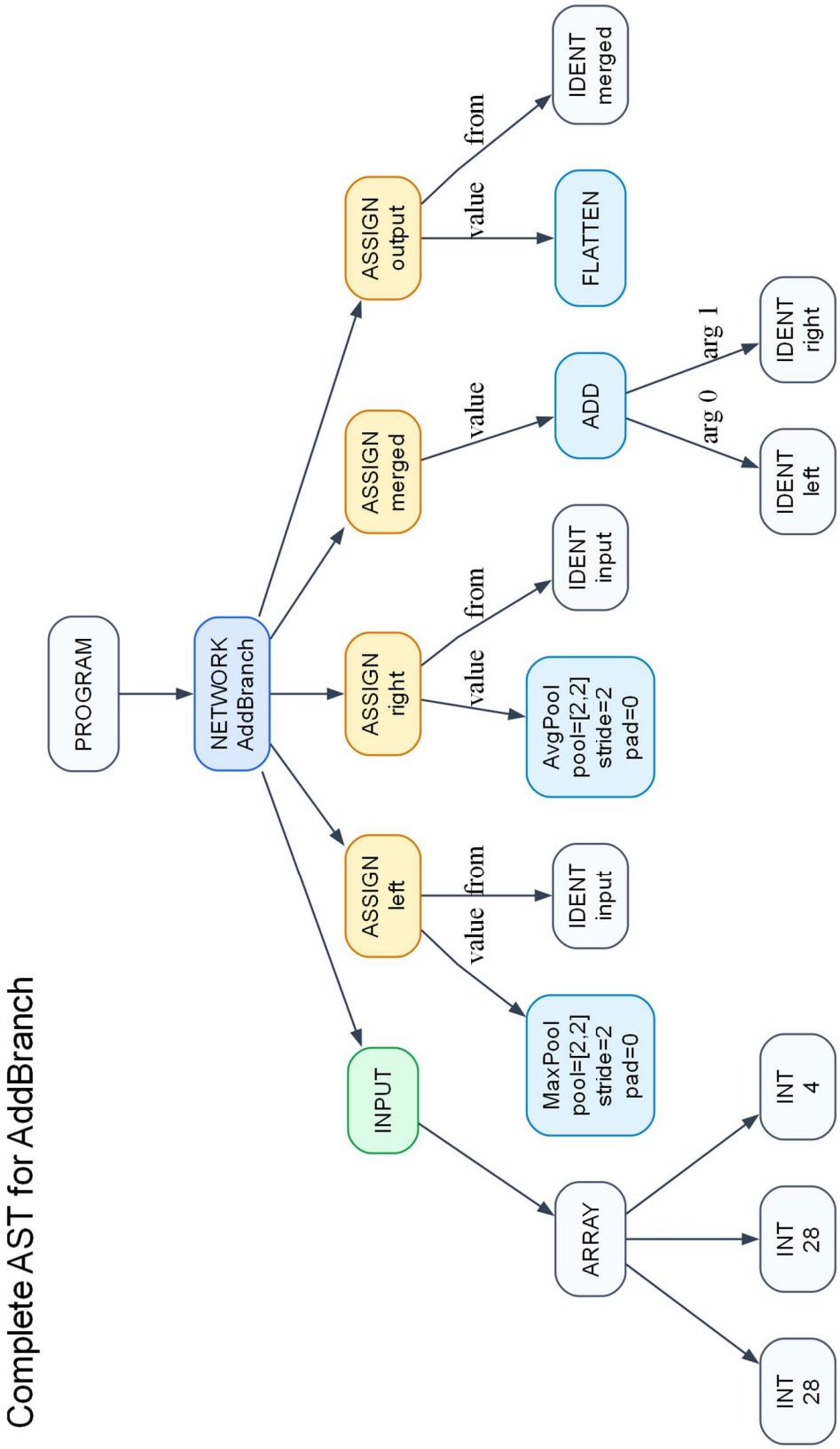

Figure 2. Complete AST for the smaller AddBranch program, including the full statement subtree structure rather than a one-level assignment sketch.

4. Semantic Analysis, Symbol Tables, and Tensor Types

Semantic analysis begins in analyze_program(ast_root). This phase creates the global scope, enters network or module scopes, inserts symbols, validates source visibility, and infers tensor shapes for the codegen-ready subset.

The symbol table is intentionally simple but essential. Scope stores the namespace hierarchy, while Symbol stores the semantic record for a name: symbol kind, attached tensor type when available, AST node reference, and source line. This phase is where the compiler turns raw identifiers into meaningful program entities.

The semantic type system is tensor-oriented rather than scalar-oriented. A TensorType stores rank and dimensions. Conv2D, pooling, Flatten, Dense, Add, and Concat each apply a different output-shape rule. This is the compiler's notion of type safety: not only must names exist, but the tensor flowing through the pipeline must satisfy the rank and dimension contract of each operator.

Again, a smaller network makes it possible to show the complete semantic derivation tree. The following figure expands the full analysis of AddBranch instead of showing only the top level of a larger network.

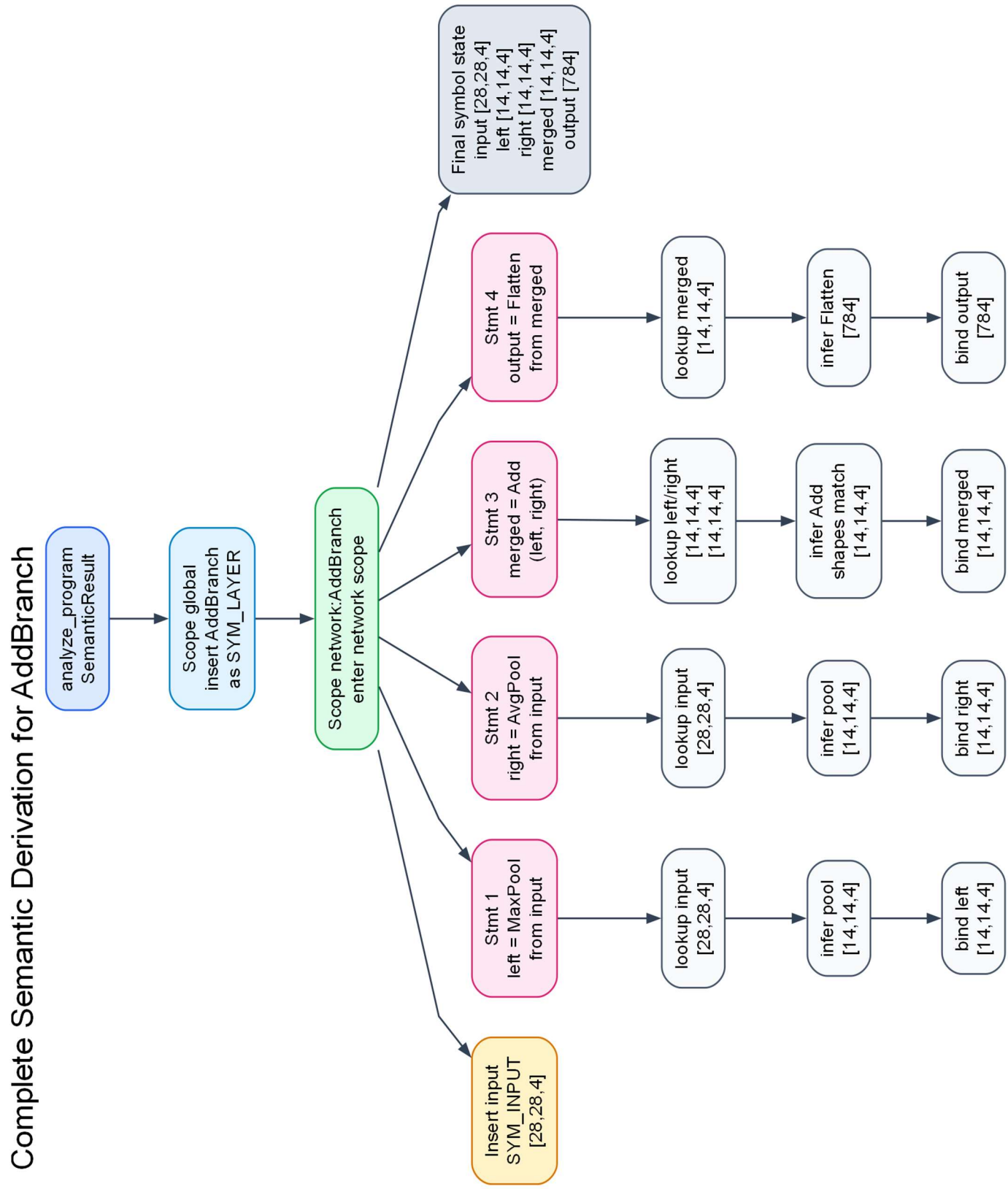

Figure 3. Complete semantic derivation for AddBranch: scope creation, lookups, tensor-shape inference, and final bindings.

- scope_lookup(scope, name, 1) is used when a name is referenced and outer scopes are visible
- scope_lookup(scope, name, 0) is used by scope_bind_variable to check only the local scope
- scope_bind_variable updates the existing local x binding instead of creating an ambiguous duplicate variable symbol
- semantic_error and semantic_warning produce phase-specific diagnostics with source lines

5. Graph Lowering and Dependency Recovery

Semantic analysis intentionally keeps source-level rebinding simple: the latest x overwrites the previous meaning of x in the current scope. That is good for source reasoning, but not sufficient for backend planning. The graph phase therefore reconstructs explicit intermediate values.

src/graph/graph.c builds a NetGraph composed of GraphNode and GraphValue objects. Each assignment creates a fresh GraphValue with a unique storage_name such as value_1, value_2, and so on. The graph also stores producer links, consumer lists, topological order, the network input type, and the chosen weight path.

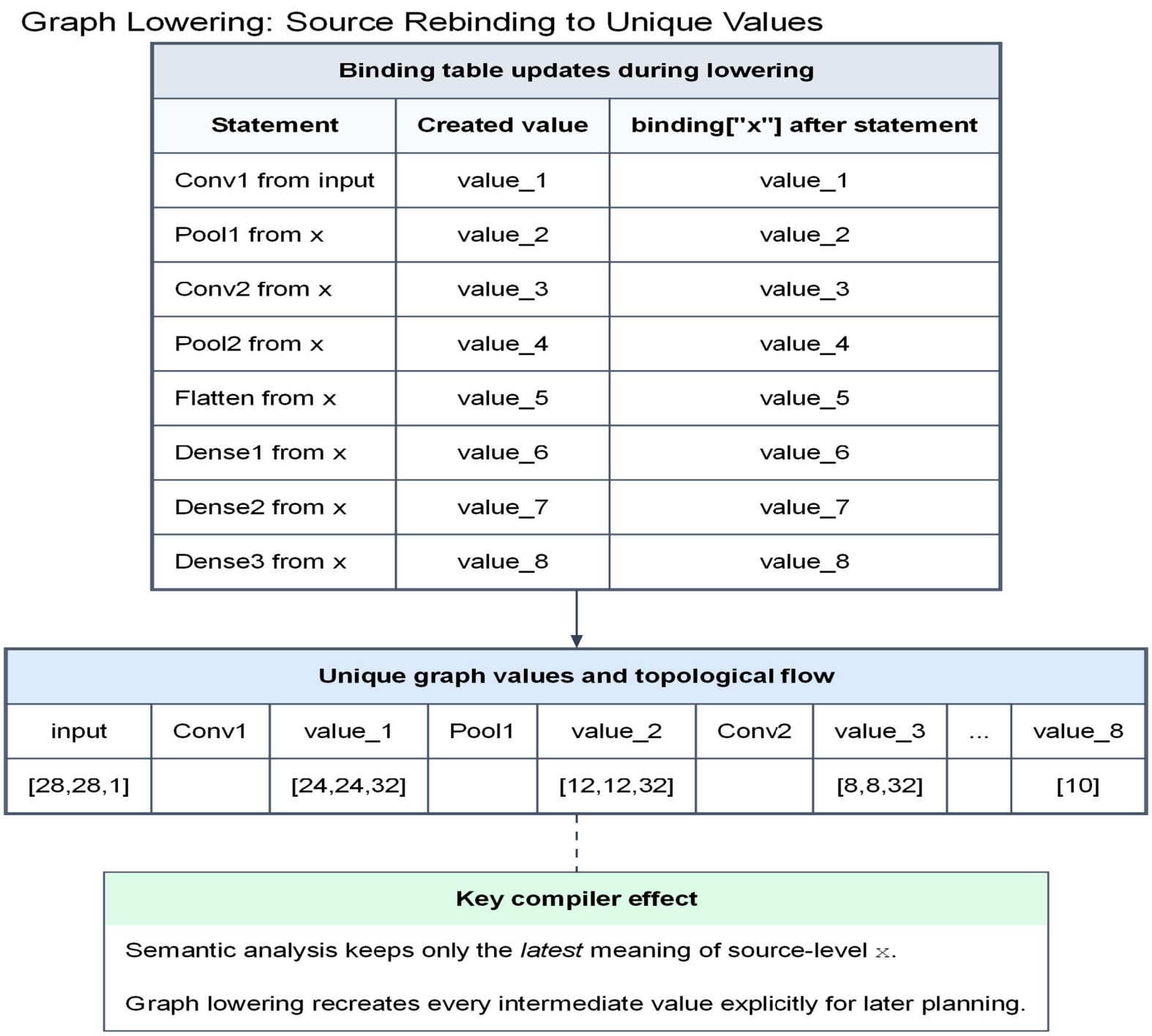

Figure 4. Graph lowering converts repeated source-level x bindings into unique values so that later passes can reason about dependencies and lifetimes precisely.

6. Planner Stack and Backend Specialization

Once a NetGraph exists, the backend does not jump directly to code emission. Instead it constructs several planner artifacts, each solving a different tensor-specific backend problem. This planner stack is the clearest architectural difference between NetLang and a traditional compiler for a general-purpose language.

LayoutPlan decides when a Flatten result can alias an existing 3D value instead of forcing an actual copy. WeightPackPlan decides which layers need runtime repacking, including Conv OC8 packing and Dense HWC-specialized packing. MemoryPlan computes value lifetimes and reusable activation slots. ConvExecutionPlan chooses the output-width micro-tile for convolution. KernelPlan selects the concrete backend kernel family per layer.

- LayoutPlan is closest to a semantic-layout bridge: it reasons about aliasing and layout transforms.
- MemoryPlan plays a role analogous to coarse-grained register allocation, but the managed objects are tensor activations rather than scalar temporaries.
- WeightPackPlan and KernelPlan have no close equivalent in a normal scalar compiler because they encode data-layout and micro-kernel specialization decisions.

7. Detailed LeNet-5 Dry Run

The LeNet-5 example is the most useful dry run because it exercises Conv2D, MaxPool, Flatten, Dense, repeated source-level rebinding of x, inline activation metadata, planner-driven weight transformations, and activation slot reuse.

```
network LeNet5 {
    input(shape: [28, 28, 1])
    weights("assets/weights/netlang/lenet5_trained.nwf")

    x = Conv2D(filters: 32, kernel: [5, 5], stride: 1, padding: 0, activation:
relu) from input
    x = MaxPool(pool: [2, 2], stride: 2, padding: 0) from x
    x = Conv2D(filters: 32, kernel: [5, 5], stride: 1, padding: 0, activation:
relu) from x
    x = MaxPool(pool: [2, 2], stride: 2, padding: 0) from x
    x = Flatten() from x
    x = Dense(units: 100, activation: relu) from x
    x = Dense(units: 100, activation: relu) from x
    x = Dense(units: 10, activation: softmax) from x
}
```

```
[1/5] Parsing examples/lenet5.nlang
      [OK] parsing successful
[2/5] Running semantic analysis
      [OK] semantic analysis passed
```

```
[3/5] Running current optimization pass
      [OK] fusion analysis completed
[4/5] Generating C code
      [OK] code generation complete
[5/5] External native compilation step follows with gcc/clang
```

The semantic stage begins by inserting input with type [28, 28, 1] into the network scope. The first convolution resolves input through scope_lookup, computes [24, 24, 32], and binds x to that shape. The next MaxPool resolves x, computes [12, 12, 32], and updates the same local x binding. This process continues until the final Dense layer leaves x with shape [10].

Graph lowering then replaces the semantic rebinding story with explicit values value_1 through value_8. This is the representation used by the planner stack. The final output value is value_8, while earlier values remain available for lifetime and dependency analysis.

The layout planner detects that the Flatten result feeds only Dense consumers and can therefore be elided. The weight planner repacks both convolution layers into OC8 form and repacks the first Dense layer to consume the aliased HWC storage directly. The memory planner discovers that only two activation slots are required for the full LeNet-5 pipeline.

Step	Source operation	Input	Output	GraphValue	Storage	Backend action
1	Conv2D(relu) from input	[28,28,1]	[24,24,32]	value_1	slot_0	packed + fused + blocked conv
2	MaxPool from x	[24,24,32]	[12,12,32]	value_2	slot_1	maxpool2d_forward
3	Conv2D(relu) from x	[12,12,32]	[8,8,32]	value_3	slot_0	packed + fused + blocked conv
4	MaxPool from x	[8,8,32]	[4,4,32]	value_4	slot_1	maxpool2d_forward
5	Flatten from x	[4,4,32]	[512]	value_5	alias(value_4)	copy elided by layout planner
6	Dense(100,relu) from x	[512]	[100]	value_6	slot_0	dense_relu_forward_avx2 with repacked HWC weights
7	Dense(100,relu) from x	[100]	[100]	value_7	slot_1	dense_relu_forward_avx2
8	Dense(10,softmax) from x	[100]	[10]	value_8	slot_0	dense_forward_avx2 + softmax_inplace

Activation-slot reuse schedule for the final generated network:

Storage	Conv1	Pool1	Conv2	Pool2	Flatten	Dense1	Dense2	Dense3
slot_0	value_1		value_3			value_6		value_8
slot_1		value_2		value_4	alias(value_5)		value_7	

```
/* Conv2D: [28,28,1] -> [24,24,32], K=5x5, S=1, P=0, plan=OWx4
[PACKED+FUSED+BLOCKED] */
conv2d_relu_packed_oc8_blocked_avx2(
    input, net->packed_conv_weights_0, conv_bias_0, net->slot_0,
    28, 28, 1,
    5, 5,
    32,
    1, 0,
    8,
    conv_spatial_block_width_0,
    net->thread_pool
);

/* Flatten elided: Dense consumer(s) read HWC storage directly via repacked
weights [512] */
dense_relu_forward_avx2(...);
```

8. How a Program Actually Becomes C: AddBranch Walkthrough

LeNet-5 is excellent for the full optimization story, but it is too large when the goal is to see the representation-by-representation conversion into emitted C. For that purpose, AddBranch is a better demonstration because it has branching, a merge operator, no weights, and a short generated C file.

```
network AddBranch {
    input(shape: [28, 28, 4])

    left = MaxPool(pool: [2, 2], stride: 2, padding: 0) from input
    right = AvgPool(pool: [2, 2], stride: 2, padding: 0) from input
    merged = Add(left, right)
    output = Flatten() from merged
}
```

The frontend parses four assignments and builds four layer AST nodes. Semantic analysis then resolves input, left, right, and merged through the symbol table while inferring [14,14,4] for the pooling outputs, [14,14,4] for the Add result, and [784] for the final Flatten output.

Graph lowering turns these source-level names into four explicit graph values: value_1 for MaxPool, value_2 for AvgPool, value_3 for Add, and value_4 for Flatten. At this point the compiler has a dependency graph with fan-out from input and fan-in at Add.

The backend then extracts LayerInfo records, builds the memory plan, and emits C. Because AddBranch contains no Conv2D or Dense layers, the generated initializer emits net->weights = NULL and no packed-weight preparation code. The emitted network state contains only an activation arena and three slots.

Source statement	Semantic fact	Graph lowering	Emitted C effect
left = MaxPool(...) from input	input : [28,28,4] -> left : [14,14,4]	value_1 = MaxPool(input)	maxpool2d_forward(input, net->slot_0, ...)
right = AvgPool(...) from input	input : [28,28,4] -> right : [14,14,4]	value_2 = AvgPool(input)	avgpool2d_forward(input, net->slot_1, ...)
merged = Add(left, right)	left/right shapes match, merged : [14,14,4]	value_3 = Add(value_1, value_2)	const float* add_inputs_2[2] = { net->slot_0, net->slot_1 }; add_forward(add_inputs_2, net->slot_2, 2, 784);
output = Flatten() from merged	merged : [14,14,4] -> output : [784]	value_4 = Flatten(value_3)	flatten_hwc_to_chw(net->slot_2, net->slot_0, 14, 14, 4); memcpy(output, net->slot_0, 784 * sizeof(float));

```
typedef struct {
    WeightFile* weights;
    NetLangThreadPool* thread_pool;
    int thread_count;
    int conv_spatial_block_override;
    float* arena;         /* 9408 bytes total */
    float* slot_0;        /* 784 floats */
    float* slot_1;        /* 784 floats */
    float* slot_2;        /* 784 floats */
} NetworkState;

/* ========== Load Weights ========== */
net->weights = NULL;

/* ========== value_1 ========== */
maxpool2d_forward(input, net->slot_0, 28, 28, 4, 2, 2, 2, 0);

/* ========== value_2 ========== */
avgpool2d_forward(input, net->slot_1, 28, 28, 4, 2, 2, 2, 0);

/* ========== value_3 ========== */
const float* add_inputs_2[2] = { net->slot_0, net->slot_1 };
add_forward(add_inputs_2, net->slot_2, 2, 784);
```

```
/* ========== value_4 ========== */
flatten_hwc_to_chw(net->slot_2, net->slot_0, 14, 14, 4);
memcpy(output, net->slot_0, 784 * sizeof(float));
```

This smaller example makes the code-generation evolution explicit. The source program does not become generic interpreter logic. It becomes a C struct specialized to the computed arena size, a fixed set of slot pointers, and a straight-line network_infer() routine whose statements mirror the topological order of the lowered graph.

9. What This Compiler Offers Beyond a Traditional Compiler

The report should be explicit here: NetLang is not better than a traditional compiler in a general sense; it is better specialized for a narrow tensor domain because it gives up a large amount of user freedom in exchange for static predictability.

A conventional compiler does not normally know that an intermediate value is a 3D image tensor of shape [24, 24, 32], that a later Dense consumes only a flattened alias of that tensor, that the corresponding weight matrix should be repacked to match that layout, or that only two activation slots are needed across the full network. NetLang does know those things because the language and compiler are built around them.

Aspect	Traditional compiler	NetLang compiler
Program model	imperative program with control flow	tensor dataflow graph described by layer assignments
Primary semantic concern	scalar types, declarations, CFG consistency	tensor rank, tensor dimension, source visibility, graph validity
Main IR family	TAC, SSA, CFG	GraphNode / GraphValue dependency graph
Allocation target	registers, stack slots, heap objects	activation arena slots and packed-weight buffers
Optimization emphasis	instruction scheduling, constant propagation, CFG transforms	flatten aliasing, weight packing, slot reuse, kernel specialization
Backend output	assembly, machine code, bytecode	specialized C linked with runtime and AVX2 kernels
User freedom	high freedom, broad semantics	restricted DSL with stronger static assumptions

- Its semantic safety story is tensor compatibility, not just scalar declaration correctness.
- Its IR is a dependency graph, not a control-flow graph.
- Its memory optimization problem is tensor-slot reuse, not register allocation in the usual sense.

- Its backend specialization problem is packed layout and kernel-family selection, not low-level instruction selection alone.
- Its emitted C is already structurally specialized for the network being compiled.

10. Diagnostics, Type Safety, and User Degree of Freedom

- Lexer diagnostics report unexpected characters with line numbers.
- Parser diagnostics report syntax failures and literal-form constraints such as invalid integer arrays.
- Semantic diagnostics report undefined variables, missing from clauses, invalid tensor ranks, unsupported codegen-subset constructs, and shape mismatches.
- Graph-lowering diagnostics report undefined source values and unsupported lowering forms.
- Code generation refuses unsupported graph node kinds that survive into backend stages.

Type safety in NetLang is domain-specific. A program is considered sufficiently well-typed for code generation only when every referenced source is visible, every operator receives an admissible tensor rank, every merge operation receives compatible dimensions, and every output shape can be inferred before backend emission begins.

The user freedom story is equally important. The language intentionally does not provide arbitrary control flow, dynamic shapes, training semantics, or unrestricted module execution. That reduction in expressiveness is exactly what allows stronger ahead-of-time reasoning about memory layout, value lifetimes, and backend specialization.

11. Current Strengths, Limitations, and Honest Positioning

- The compiler already has a real multi-stage pipeline rather than a single-pass translator.
- The graph phase and planner stack are genuine architectural contributions for a course project.
- The LeNet-5 path demonstrates real specialization: flatten elision, weight repacking, slot reuse, and packed fused blocked kernels.
- The fusion pass file is only partially wired; practical fusion currently comes mostly from backend kernel choice on inline activations.
- BatchNorm, LayerNorm, and module calls are parsed but not part of the codegen-ready subset.
- The target domain remains fixed-shape inference, not training and not general dynamic graph execution.

The best final narrative is therefore both ambitious and honest: this project implements a technically coherent compiler for a restricted CNN DSL, and its distinctive value lies in tensor-aware static analysis and planner-driven backend specialization rather than in breadth of language support.

12. Conclusion

The NetLang compiler should be presented as a tensor-aware, planner-driven compiler pipeline. Its frontend parses a deliberately restricted neural-network DSL. Its semantic phase resolves names and

tensor shapes. Its graph phase recovers explicit data dependencies from source-level rebinding. Its planner stack computes layout, weight, memory, execution, and kernel decisions. Its backend emits specialized C rather than a generic interpreter-style execution plan.

The LeNet-5 dry run is the most convincing proof of value. A short sequence of source assignments becomes a graph of unique values, a compact two-slot activation arena, repacked weight buffers, and backend calls that are visibly fused, blocked, and layout-sensitive. That is the specific compiler story this project can defend strongly in a technical report.